

Common Types

Purpose: Define reusable data structures that appear across multiple metamodel layers. These types represent fundamental concepts that transcend the layer boundaries and provide structural consistency throughout the model.

Overview

Common Types are transversal structures used throughout the HAROS2 metamodel. Unlike layer-specific entities, these types:

- **Cross layer boundaries**, seeing as they are used in Layer 0 through Layer 6.
- **Provide structural consistency** with standard formats for source references, provenance tracking, etc..
- **Enable composition** of complex entities, by providing useful primitives.
- **Simplify implementation** by sharing validation and serialization logic.

SourceRef

Represents a specific location in a source file. Any entity that can be traced to source code includes an optional **SourceRef**.

Fields

Field	Type	Required	Description
file_path	string	Yes	Path to source file (relative to workspace or package root)
line_start	int	Yes	Starting line number (1-indexed)
line_end	int	No	Ending line number (inclusive), if multi-line
column_start	int	No	Starting column (1-indexed)
column_end	int	No	Ending column (inclusive)
note	string	No	Human-readable note explaining relevance

Design Rationale

- **1-indexed:** Matches most editor and compiler conventions
- **Optional ranges:** Balance precision with complexity
- **Relative paths:** Workspace-relative for portability
- **Note field:** Adds human context to technical location

Validation Rules

1. **file_path** must not be empty

- 2. `line_start` ≥ 1
- 3. If `line_end` specified: `line_end` $\geq$ `line_start`
- 4. If columns specified on same line: `column_end` $\geq$ `column_start`
- 5. `file_path` should reference existing Layer 0 File (validation warning if not)

Example: Coarse (File-level)

```
{
  "file_path": "src/sensor_node.cpp"
}
```

Example: With Context Note

```
{
  "file_path": "src/camera_node.cpp",
  "line_start": 45,
  "note": "Publisher created with best_effort reliability"
}
```

ElementProvenance

Represents how an entity was discovered or derived. It should be attached to metamodel entities to document extraction method, confidence, and context. Critical for understanding model quality and limitations.

Fields

Field	Type	Required	Description
<code>extraction_method</code>	enum	Yes	How entity was discovered
<code>confidence</code>	float	Yes	Confidence in correctness (0.0-1.0)
<code>source_location</code>	SourceRef	No	Primary source location
<code>additional_locations</code>	list[SourceRef]	No	Secondary locations (for composed entities)
<code>extractor_version</code>	string	No	Version of analysis tool
<code>extraction_timestamp</code>	string	No	When extraction occurred (ISO 8601)
<code>extraction_context</code>	map[string, any]	No	Additional context
<code>notes</code>	string	No	Human-readable explanation

Extraction Methods

1. **static_analysis**: Entity extracted by analyzing source code without execution.
2. **runtime_discovery**: Entity observed in running system through introspection.
3. **annotation**: Entity manually specified by user or domain expert.
4. **composition**: Entity derived by combining other entities.
5. **heuristic**: Entity inferred using pattern matching without definitive evidence.

Confidence Scale

- **1.0 (Certain)**: Observed at runtime or ground truth
- **0.9-0.99 (Very High)**: Direct static analysis of clear patterns
- **0.7-0.89 (High)**: Static analysis with moderate complexity
- **0.5-0.69 (Medium)**: Complex control flow or dataflow analysis
- **0.3-0.49 (Low)**: Heuristic-based inference
- **< 0.3 (Very Low)**: Speculative guesses

Validation Rules

1. **extraction_method** must be recognized enum value
2. **confidence** must be in [0.0, 1.0]
3. **extraction_timestamp** must be valid ISO 8601 if specified
4. **source_location** must be valid SourceRef if specified

Example: Static Analysis

```
{
  "extraction_method": "static_analysis",
  "confidence": 0.95,
  "source_location": {
    "file_path": "src/camera_node.cpp",
    "line_start": 78
  },
  "extractor_version": "haros-cpp-analyzer-2.0.1",
  "extraction_context": {
    "parser": "clang",
    "analysis_depth": "control_flow"
  }
}
```

Example: Runtime Discovery

```
{
  "extraction_method": "runtime_discovery",
  "confidence": 1.0,
  "extraction_timestamp": "2025-12-08T14:30:00Z",
  "extraction_context": {
    "discovery_tool": "ros2 node list",
    "system_uptime": 120.5
  }
}
```

```
}  
}
```

QoSProfile

Complete specification of Quality of Service policies for a publisher or subscriber. It defines communication behavior and compatibility constraints.

Fields

Field	Type	Required	Description
<code>history</code>	enum	Yes	History policy: <code>keep_last</code> or <code>keep_all</code>
<code>depth</code>	int	Conditional	Queue depth (required if <code>history == "keep_last"</code>)
<code>reliability</code>	enum	Yes	Reliability policy: <code>reliable</code> or <code>best_effort</code>
<code>durability</code>	enum	Yes	Durability policy: <code>transient_local</code> or <code>volatile</code>
<code>deadline</code>	Duration	No	Maximum time between messages
<code>lifespan</code>	Duration	No	Maximum message age
<code>liveliness</code>	enum	No	Liveliness policy: <code>automatic</code> , <code>manual_by_topic</code> , <code>manual_by_node</code>
<code>liveliness_lease_duration</code>	Duration	No	Liveliness timeout period
<code>avoid_ros_namespace_conventions</code>	bool	No	Whether to bypass ROS 2 namespace remapping

Validation Rules

1. `history` must be `"keep_last"` or `"keep_all"`
2. If `history == "keep_last"`, `depth` must be specified and > 0
3. `reliability` must be `"reliable"` or `"best_effort"`
4. `durability` must be `"transient_local"` or `"volatile"`
5. All Duration fields must be positive if specified
6. `liveliness` must be one of defined enum values if specified

Example

```
{
  "history": "keep_last",
  "depth": 10,
  "reliability": "reliable",
  "durability": "volatile",
  "deadline": {"seconds": 0.1}
}
```

Duration

Represents a time interval or period. Used for timer periods, QoS deadlines, callback execution times, timeouts, and any time-based measurement.

Fields

Field	Type	Required	Description
seconds	float	Conditional	Duration in seconds (mutually exclusive with nanoseconds)
nanoseconds	int	Conditional	Duration in nanoseconds (mutually exclusive with seconds)

Design Rationale

- **Dual representation:** Supports both high-precision (nanoseconds) and human-readable (seconds)
- **Single-field requirement:** Prevents ambiguity and inconsistency
- **Precision preservation:** Nanoseconds avoid floating-point rounding for exact values
- **Readability:** Seconds more intuitive for configuration and documentation

Validation Rules

1. Exactly one of `seconds` or `nanoseconds` must be specified
2. Values must be positive (> 0)
3. `nanoseconds` must be integer
4. `seconds` may be float

Example

100 milliseconds (as seconds):

```
{"seconds": 0.1}
```

100 milliseconds (as nanoseconds):

```
{"nanoseconds": 100000000}
```

ElementRef

Represents a reference to another metamodel entity. Used in findings, analysis results, relationships, and any cross-entity reference where a full entity object is not appropriate.

Fields

Field	Type	Required	Description
type	string	Yes	Entity type (e.g., "node", "publication", "topic")
id	string	Yes	Unique identifier of the referenced entity
role	string	No	Role in context (e.g., "source", "target", "publisher")

Valid Entity Types

Type	Layer	Description
file	0	Source file
package	0	ROS 2 package
node_implementation	1	Node class definition
message_definition	1	Message type
launch_description	2	Launch configuration
feature	3	Variability feature
configuration	3	Feature selection
node	4	Runtime node instance
topic	4	Communication topic
service	4	Service interface
action	4	Action interface
publication	5	Publisher endpoint
subscription	5	Subscriber endpoint
timer	4	Periodic timer
finding	6	Analysis finding
metric	6	Computed metric

Validation Rules

1. type must be a recognized entity type.
2. id should reference an existing entity of that type (validation warning if not found).
3. role is optional and context-dependent.

Example

```
{
  "type": "publication",
  "id": "pub_camera_image",
  "role": "publisher"
}
```

Usage Guidelines

When to Use SourceRef

✅ Use SourceRef when:

- Entity corresponds to specific code location
- Traceability to source is valuable
- IDE navigation is desired
- Manual inspection may be needed

❌ Don't use SourceRef when:

- Entity is purely derived (no single source location)
- Entity is from runtime discovery (no source code)
- Overhead not justified (bulk generated entities)

When to Include Provenance

✅ Include provenance for:

- All extracted entities
- Runtime-discovered entities
- Derived entities where method matters

❌ Omit provenance for:

- Layer 0 entities (files are self-evident)
- User-provided configurations (always ground truth)
- Simple lookups or trivial derivations

QoS Profile Defaults

When QoS is not explicitly specified, use standard ROS 2 defaults:

```
{
  "history": "keep_last",
  "depth": 10,
  "reliability": "reliable",
}
```

```
"durability": "volatile"  
}
```

Document when defaults are applied vs. explicitly specified.

Design Rationale Summary

Why Common Types?

1. **Consistency**: Same structure for same concept everywhere
2. **Reusability**: Define once, validate once, serialize once
3. **Maintainability**: Changes to common types propagate automatically
4. **Documentation**: Single source of truth for cross-cutting concerns

Why These Specific Types?

- **SourceRef**: Traceability is fundamental to HAROS2's value proposition
- **ElementProvenance**: Transparency about extraction quality is critical for trust
- **QoSProfile**: Central to ROS 2 communication, appears in multiple contexts
- **Duration**: Time is pervasive (timers, deadlines, latencies)
- **ElementRef**: Lightweight references avoid data duplication and circular dependencies

Trade-offs

-  **Simplicity**: Common types are relatively simple structures
-  **Flexibility**: Optional fields accommodate varied usage contexts
-  **Verbosity**: Full provenance on every entity can be verbose (mitigated by omitting when unnecessary)
-  **Validation complexity**: Cross-references require multi-pass validation